



Editing BIPL, Properly

A Language Server and VS Code Client for the
Basic Imperative Programming Language of YAS

```
{  
  y = 1;  
  i = 1;  
  while (i <= x) {  
    ~ x: program input  
    y = y * i;  
    i = i + 1;  
  }  
}
```

*factorialV1.bipl — an official YAS sample, as the server
sees it*

SLE Assignment 3 — Option LSP.2

Vivekananda Reddy Godala

Software Language Engineering · Prof. Ralf Lämmel · 2026-07-13

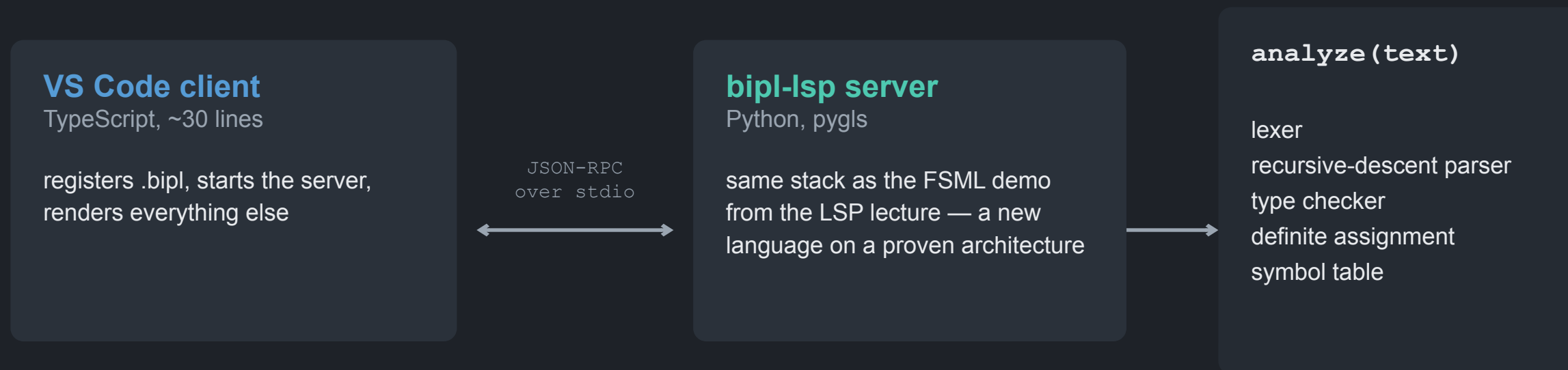
BIPL in 30 seconds

```
// Euclidean division
{
  q = 0;
  r = x;
  while (r >= y) {
    r = r - y;
    q = q + 1;
  }
  if (r < 0)
    r = -r;
}
```

A tiny fragment of C: assignment, blocks, if/else, while; integer and boolean expressions. No declarations, no I/O, no procedures.

- **No declarations** — a variable's type exists only implicitly — the editor must infer it
- **No boolean literals** — bool values arise only from comparisons and `&&`, `||`, `!`
- **Inputs come from the initial store** — official samples read variables they never assign (“Assume x to be positive”)

Architecture: one analysis, many features



Everything is one pure function: `analyze(text) → AST + diagnostics + symbol table`. Hover, definition, references, rename, completion, and outline are all cheap queries against its result — cached per document version.

The static semantics behind the squiggles

Type inference (no declarations!)

A variable gets its type from its first assignment, following YAS's TypeChecker.hs:

```
x = 5;           // x : int
x = 1 < 2;       // error: int vs bool
```

Input variables are typed from use — the demand-driven twist this project adds:

```
while (i <= x) // x : int
if (b)         // b : bool
```

Definite assignment (data flow)

A read is safe only if the variable is assigned on every path that reaches it:

```
if (c) y = 1;
z = y;  // warning: maybe

while (c) v = 1;
u = v;   // body may run 0×
```

`if` counts only what both branches assign; a `while` body counts for nothing. Assignment on both branches is definite.

Not every unassigned read is an error

BIPL programs receive inputs through the initial store. Official YAS samples read variables they never assign — factorialV1.bipl literally says “*Assume x to be positive*”. A naive “use before assignment” error would flag the language's own samples as broken.



Error

`type-error`

conditions must be boolean; operands must match; a variable keeps its inferred type



Warning

`maybe-uninitialized`

assigned somewhere, but not on every path reaching this read



Information

`input-variable`

never assigned at all → treated as a program input; type inferred from use

Consequence: all 12 official YAS samples analyze error-free — and the test suite enforces that as a regression check.

LSP features, and what powers them

Diagnostics	<code>publishDiagnostics</code>	parser recovery + type checker + definite assignment
Hover	<code>textDocument/hover</code>	inferred type, input status, assignment/read counts
Go to definition	<code>textDocument/definition</code>	a variable's first assignment
Find references / Rename	<code>references</code> / <code>rename</code>	all occurrences from the symbol table (rename validates the new identifier)
Completion	<code>textDocument/completion</code>	keywords + known variables, each labeled with its inferred type
Outline	<code>documentSymbol</code>	variables with types in the breadcrumb/outline view
Quick fix	<code>codeAction</code>	initialize a maybe-uninitialized variable at program start

Every feature is a query against the cached result of `analyze(text)` — no feature has its own parser.

Demo

- 1** `good.bipl` Euclidean division — blue “program input” hints on x and y; hover shows types inferred purely from use
- 2** `bad.bipl` five diagnostics: three type errors, two data-flow warnings — real analyses, not keyword matching
- 3** `Navigate` hover a variable · Cmd-click to its first assignment · find all references
- 4** `Refactor` F2 rename (rejects invalid identifiers) · quick-fix a maybe-uninitialized warning
- 5** `Live typing` break the factorial sample and watch the diagnostics move as the analysis re-runs on every keystroke



What I'd want you to remember

LSP decouples smartness from the editor

one Python analysis serves any LSP editor; the VS Code client is ~30 lines

Small language ≠ trivial tooling

even 5 statement forms force real decisions: inference without declarations, branch-sensitive data flow

The language's own corpus is the spec

YAS samples decided the input-variable design — and now guard it as tests

Code, tests, and demo files: github.com/VivekanandaReddy3/sle-bipl-lsp